

On the Approximation of Koopman Eigenfunctions using Random Features

Chunyang Liao*

Abstract. Nonlinear dynamical systems are ubiquitous in various scientific areas. The Koopman operator theory provides a way to analyze nonlinear dynamical systems with a linear operator, whose spectra contain rich information in understanding the behaviors of dynamical systems. In this paper, we aim to use data-driven method to construct the principal eigenfunctions of the Koopman operator. Our approach relies on the decomposition of Koopman eigenfunction where the linear part associates with the left eigenvectors of the linearization of the system at the equilibrium and the nonlinear part satisfies a linear PDE. Then we solve the linear PDE using random features model, which is a data-driven machine learning technique for large-scale kernel approximation. In addition, the random features model can be viewed as a randomized shallow neural network and only the linear readout needs to train, which significantly reduces the computational complexity. We provide a computational framework for the approximation of Koopman eigenfunctions and an error analysis for our proposed method. Finally, we present simulation results for learning the principal eigenfunctions and constructing the Lyapunov functions. Numerical experiments indicate that our method is fast and accurate. As such, it can serve as a benchmark for approximating Koopman eigenfunctions.

Key words. Dynamical system, Koopman operator, Koopman eigenfunctions, Random features

MSC codes. 37M99, 41A99, 65C99, 65P99

1. Introduction. Dynamical system theory provides a solid mathematical framework for analyzing and understanding how systems evolve with time. It is frequently applied to time series data, which are ubiquitous in various scientific domains. Traditionally, dynamical systems are represented in the immediate state space, which contains all possible states that the systems can be in. The seminal work [19] introduced Koopman operator theory, which is named after B.O.Koopman - the author of the fundamental work [18] on transformations of Hamiltonian systems in Hilbert space. Koopman operator approximates a nonlinear dynamical system by a linear one that maps the nonlinear evolution of a finite-dimensional state space to an infinite-dimensional but linear functional framework. Koopman operator theory has emerged as an important tool to study dynamical systems over the past decade, see the review paper [5] and references therein.

The spectrum of Koopman operator plays a significant role in theoretical analysis. For example, the main results of [27] established the relationship between the existence of specific eigenfunctions of the Koopman operator and the global stability property of hyperbolic equilibrium points and limit cycles. In [40], the authors characterized the stable manifold in terms of the zero-level curve of the Koopman eigenfunction. Moreover, one can use Koopman eigenfunctions to construct solutions to the optimal control problems [41, 42]. Koopman eigenfunctions can also be used to construct Lyapunov functions for globally asymptotically stable dynamics [39, 23]. However, the computation of Koopman operator faces challenges because Koopman operator can have both discrete and continuous spectra [29] and it suffers from "spectra pollution" when approximating the spectra of an infinite-dimensional operator

*Department of Mathematics, University of California, Los Angeles, CA liao-chunyang@math.ucla.edu.

via a finite-dimensional matrix [11]. The goal of this paper is to develop a data-driven method to construct the eigenfunctions of the Koopman operator.

Traditional numerical methods approximate Koopman operator from data first and then derive its spectra [17]. Nevertheless, those method may lead to spurious eigenvalues due to noise or unstable numerical method [15, 9, 14], which hinders representing underlying dynamics and analyzing system’s behavior. In the paper, we propose a data-driven method to learn the Koopman eigenfunctions directly without learning Koopman operator itself. Our approach relies on the decomposition of the principal eigenfunctions in [12]. Precisely, the principal eigenfunctions can be decomposed into linear and nonlinear components, where the linear component corresponds to the eigenvector of local linearization at the origin and nonlinear part is the solution of a partial differential equation (PDE). To solve the PDE corresponding to the nonlinear part, a path-integral formula based approach and a deep neural network approach were proposed in [12]. In [22], the authors applied kernel method to solve PDEs in reproducing kernel Hilbert spaces (RKHS). They also provide an error estimate for their proposed kernel-based method. Building upon the theory of RKHS, kernel-based methods have been widely used in machine learning [37], solving PDEs [7, 2], and operator learning [43, 10, 3]. However, the kernel method is not computationally efficient when the sample size is large. Specifically, kernel method requires $\mathcal{O}(m^3)$ training time and $\mathcal{O}(m^2)$ to store the kernel matrix, where m is the sample size.

To overcome the computational bottleneck in kernel method, random features method was proposed to approximate large-scale kernel machines [33]. The main idea behind random features method is mapping data into a low-dimensional randomized feature space, which reduces the computational and storage costs of operating on kernel method. The generalization analysis of random features method has been of recent interest under regression setting [35, 28, 8] or classification setting [38]. In recent years, random features method was applied to solve partial differential equations [6, 24] and operator learning problems [32, 25]. Motivated by the great success of random features in scientific machine learning, we propose to solve the PDEs corresponding to nonlinear components of Koopman eigenfunctions decomposition by using random features method.

The reminder of this paper is organized as follows: In Section 2, we provide a brief overview of Koopman operator and its eigenfunctions. In Section 3, we provide background on random features method. In Section 4, we introduce the computational framework for the approximation of Koopman eigenfunction and its application to approximating the Lyapunov functions. In Section 5, we provide error estimates for our method and Section 6 shows some numerical experiments to test the performance of our proposed method.

1.1. Notation. Let $\mathbb{R}$ and $\mathbb{C}$ be the set of real numbers and complex numbers, respectively. Denote $i = \sqrt{-1}$ the imaginary unit and $\bar{x}$ the complex conjugate of complex number x . We also let $\mathbb{N}_+$ be all positive integers. Notation $[N]$ represents all positive integers smaller than N , i.e. $[N] = 1, 2, \dots, N$. We use boldfaced lower-case Latin letters to denote vectors (i.e. $\mathbf{x}$). For any vector $\mathbf{x} \in \mathbb{C}^d$, we use x_j to denote its j -th entry. For any vector $\mathbf{x}, \mathbf{y} \in \mathbb{C}^d$, the dot product is denoted by $\mathbf{x} \cdot \mathbf{y} = \sum_{j=1}^d x_j \bar{y}_j$, which is the same as inner product $\langle \mathbf{x}, \mathbf{y} \rangle$. Denote $\|\mathbf{x}\|_2$ the ℓ_2 norm of vector $\mathbf{x}$. We use boldfaced upper-case Latin letters to denote matrices (i.e. $\mathbf{A}$). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a multivariate scalar-valued function, we use $\nabla f(\mathbf{x})$ to denote its

84 gradient at point $\mathbf{x} \in \mathbb{R}^d$. Let $f : \mathbb{R}^d \rightarrow \mathbb{R}^n$ be a vector-valued function, we use $\frac{\partial f}{\partial \mathbf{x}}$ to denote
 85 the Jacobian matrix, i.e. $\left(\frac{\partial f}{\partial \mathbf{x}}\right)_{j,k} = \frac{\partial f_j}{\partial x_k}$. We denote by $H^s(D)$ Sobolev space over domain D .

86 **2. Koopman Operator Theory.** In this section, we provide a brief overview of Koopman
 87 operator theory. We refer readers to [30] for more details about Koopman operator spectral
 88 theory and to [12] for decomposition of Koopman eigenfunctions.

89 **2.1. Koopman operator and its spectra.** Consider the continuous-time dynamical system

$$90 \quad (2.1) \quad \dot{\mathbf{x}} = f(\mathbf{x}), \quad \mathbf{x} \in X \subset \mathbb{R}^d,$$

91 where X is a state space. Throughout this paper, we assume that the vector field $f(\mathbf{x})$ is
 92 smooth and known to us. Let C^0 denote the continuous function from state space X to $\mathbb{C}$
 93 and let $\mathcal{F} \subset C^0$ be a (Banach) space of *observables* $\psi : X \rightarrow \mathbb{C}$. The definitions of Koopman
 94 operator and its eigenpairs are given as follows:

95 **Definition 2.1 (Koopman Operator).** Let $s_t(\mathbf{x})$ be the solution of the dynamical system (2.1)
 96 at time t starting from the initial condition $\mathbf{x}$. The family of Koopman operators $\mathbb{U}_t : \mathcal{F} \rightarrow \mathcal{F}$
 97 corresponding to (2.1) is defined as

$$98 \quad (2.2) \quad [\mathbb{U}_t \psi](\mathbf{x}) = \psi(s_t(\mathbf{x})),$$

99 where $\psi \in \mathcal{F}$ is referred to as an observable function. The infinitesimal generator $\mathcal{K}_f$ of
 100 Koopman operator $\mathbb{U}_t$ is defined as

$$101 \quad (2.3) \quad \mathcal{K}_f \psi = \lim_{t \rightarrow 0} \frac{(\mathbb{U}_t - I)\psi}{t}, \quad t \geq 0.$$

102 Furthermore, if observable function ψ is continuously differentiable, then $g(\mathbf{x}, t) = [\mathbb{U}_t \psi](\mathbf{x})$ is
 103 the solution of partial differential equation [21]

$$104 \quad (2.4) \quad \frac{\partial g}{\partial t} = \nabla g(\mathbf{x}) \cdot f(\mathbf{x})$$

105 with the initial condition $g(\mathbf{x}, 0) = \psi(x)$. While the dynamical system (2.1) can be nonlinear,
 106 a remarkable fact is that the corresponding Koopman operator (2.2) is a linear operator, i.e.
 107 $\mathbb{U}_t(a\psi_1 + b\psi_2) = a\mathbb{U}_t\psi_1 + b\mathbb{U}_t\psi_2$ for any $\psi_1, \psi_2 \in \mathcal{F}$ and $a, b \in \mathbb{R}$.

108 **Definition 2.2 (Eigenvalues and Eigenfunctions).** A function $\psi_\lambda(\mathbf{x})$, assumed to be at least
 109 C^1 , is said to be an eigenfunction of the Koopman operator associated with eigenvalue λ if

$$110 \quad (2.5) \quad [\mathbb{U}_t \psi_\lambda](\mathbf{x}) = e^{\lambda t} \psi_\lambda(\mathbf{x}).$$

111 Using Koopman generator (2.3) and PDE (2.4), the equation (2.5) can be written as

$$112 \quad (2.6) \quad \nabla \psi_\lambda(\mathbf{x}) \cdot f(\mathbf{x}) = \lambda \psi_\lambda(\mathbf{x})$$

113 In this paper, we are interested in computing the eigenfunctions associated with the eigen-
 114 values of the linearization of the nonlinear dynamical system at an equilibrium point. With

the assumption that a equilibrium point is hyperbolic, the spectrum of our interest is well-defined. Notice that equations (2.5) and (2.6) provide a "general" definition of Koopman spectrum in the sense that it holds for all $t \geq 0$ and $\mathbf{x} \in X$. However, the spectrum can be defined over finite time or over a subset of the state space. The spectrum of interest to us in this paper could be well-defined over the subset of the state space.

Definition 2.3 (Open Eigenfunction). Let $\psi : \mathcal{C} \rightarrow \mathbb{C}$, where $\mathcal{C}$ is not an invariant set. Let $\mathbf{x} \in \mathcal{C}$, and $\tau \in (\tau^-(\mathbf{x}), \tau^+(\mathbf{x})) = I_{\mathbf{x}}$ a connected open interval such that $s_{\tau}(\mathbf{x}) \in \mathcal{C}$ for all $\tau \in I_{\mathbf{x}}$. If

$$(2.7) \quad [\mathbb{U}_{\tau}\psi_{\lambda}](\mathbf{x}) = \psi_{\lambda}(s_{\tau}(\mathbf{x})) = e^{\lambda\tau}\psi_{\lambda}(\mathbf{x}), \quad \text{for all } \tau \in I_{\mathbf{x}},$$

then ψ is called an open eigenfunction of the Koopman operator family $\mathbb{U}_t$, for $t \in \mathbb{R}$ with eigenvalue λ .

If $\mathcal{C}$ is a proper invariant subset of X in which case $I_{\mathbf{x}} = \mathbb{R}$ for every $\mathbf{x} \in \mathcal{C}$, then ψ_{λ} is called the subdomain eigenfunction. If $\mathcal{C} = X$, then ψ_{λ} will be the ordinary eigenfunction associated with eigenvalue λ as defined in (2.5). When $\mathcal{C}$ is open, then the open eigenfunctions as defined above can be extended from $\mathcal{C}$ to a larger reachable set from the closure of $\mathcal{C}$, based on the construction procedure outlined in [30, Definition 5.2, Lemma 5.1]. Let D be that larger domain. With the assumption that $\mathbf{x} = 0$ is a hyperbolic equilibrium point of the system, the eigenvalues of the linearization of the system dynamics at the origin, i.e. $\mathbf{E} := \frac{\partial f}{\partial \mathbf{x}}(0)$, will form the eigenvalues of the Koopman operator [30, Proposition 5.8]. We will refer to the corresponding eigenfunctions as principal eigenfunctions and our goal is to numerically compute these eigenfunctions which are defined over domain D . Furthermore, the principal eigenfunctions are related to the famous Hartman-Grobman theorem, which transforms the nonlinear system into a linear system in a small neighborhood around the equilibrium point [20]. The principal eigenfunctions can be defined over domain D or over the entire state space $\mathbf{X}$ [30, Theorem 5.6].

2.2. Decomposition of Koopman eigenfunctions. Following [12], we consider the decomposition of nonlinear dynamical system into linear and nonlinear components

$$(2.8) \quad \dot{\mathbf{x}} = f(\mathbf{x}) = \mathbf{E}\mathbf{x} + (f(\mathbf{x}) - \mathbf{E}\mathbf{x}) := \mathbf{E}\mathbf{x} + G(\mathbf{x}),$$

where $\mathbf{E} := \frac{\partial f}{\partial \mathbf{x}}(0)$ with $\mathbf{E}\mathbf{x}$ the linear part and $G(\mathbf{x})$ the purely nonlinear part. Let λ be an eigenvalue of $\mathbf{E}$ and let $\psi_{\lambda}(\mathbf{x})$ be the eigenfunction corresponding to λ (note that it is called principal eigenfunction as introduced above). Similar to the decomposition of dynamical system into linear and nonlinear parts, the principal eigenfunction $\psi_{\lambda}(\mathbf{x})$ can also be decomposed into linear and nonlinear parts, i.e.

$$(2.9) \quad \psi_{\lambda}(\mathbf{x}) = \mathbf{w}^{\top}\mathbf{x} + h(\mathbf{x}),$$

where $\mathbf{w}^{\top}\mathbf{x}$ and $h(\mathbf{x})$ are the linear part and purely nonlinear part, respectively. Substituting (2.9) to (2.6) and comparing terms yield

$$(2.10) \quad \mathbf{w}^{\top}\mathbf{E} = \lambda\mathbf{w}^{\top}, \quad \nabla h(\mathbf{x}) \cdot f(\mathbf{x}) - \lambda h(\mathbf{x}) + \mathbf{w}^{\top}G(\mathbf{x}) = 0.$$

Therefore, the linear part $\mathbf{w}^{\top}$ is the left eigenvalue of $\mathbf{E}$ with eigenvalue λ and the nonlinear part $h(\mathbf{x})$ satisfies the linear partial differential equation.

3. Random Features. In this section, we provide background on random features. We first summarize kernel method and the theory of reproducing kernel Hilbert spaces (RKHS) in Section 3.1. In Section 3.2, we present the theory of random features.

3.1. Kernel Method and Reproducing Kernel Hilbert Space. The usage of kernel method was built upon the theory of reproducing kernel Hilbert spaces developed by N.Aronszajn [1]. In particular, Let $\mathcal{H}$ be a Hilbert space of functions defined over set $\mathcal{X}$. Denote $\langle f, g \rangle_{\mathcal{H}}$ the inner product on $\mathcal{H}$ and let $\|f\|_{\mathcal{H}} = \langle f, f \rangle_{\mathcal{H}}^{1/2}$ be the norm in $\mathcal{H}$, for f and $g \in \mathcal{H}$. We say that $\mathcal{H}$ is a reproducing kernel Hilbert space (RKHS) if every point evaluation is a bounded linear functional, i.e. there exists some $M_{\mathbf{x}} \geq 0$ such that

$$(3.1) \quad |f(\mathbf{x})| \leq M_{\mathbf{x}} \|f\|_{\mathcal{H}}, \quad \text{for all } f \in \mathcal{H}.$$

Following the Riesz representation theorem, there exists a function $K_{\mathbf{x}} \in \mathcal{H}$ such that

$$(3.2) \quad f(\mathbf{x}) = \langle f, K_{\mathbf{x}} \rangle_{\mathcal{H}}, \quad \text{for all } f \in \mathcal{H}.$$

Since every point evaluation is a bounded linear functional and we apply Riesz representation theorem one more time, then there exists a function $K_{\mathbf{x}'} \in \mathcal{H}$ such that

$$(3.3) \quad K_{\mathbf{x}}(\mathbf{x}') = \langle K_{\mathbf{x}}, K_{\mathbf{x}'} \rangle.$$

Therefore, we obtain a kernel function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ such that $K(\mathbf{x}, \mathbf{x}') = \langle K_{\mathbf{x}}, K_{\mathbf{x}'} \rangle_{\mathcal{H}}$, which is called a reproducing kernel of $\mathcal{H}$. In the rest of paper, we will use $\mathcal{H}_K$ to denote the RKHS $\mathcal{H}$ with reproducing kernel K where the dependence is explicitly stated. A reproducing kernel K of $\mathcal{H}_K$ is positive definite in the sense that

$$(3.4) \quad \sum_{i,j=1}^q a_i a_j K(\mathbf{x}_i, \mathbf{x}_j) \geq 0, \quad \text{for all } a_i \in \mathbb{R}, \mathbf{x}_i \in \mathcal{X}, \text{ and } q \in \mathbb{N}_+.$$

Common examples of reproducing kernels defined on a compact domain $\mathcal{X} \subset \mathbb{R}^d$ are

- Gaussian kernel: $K(\mathbf{x}, \mathbf{x}') = e^{-\|\mathbf{x} - \mathbf{x}'\|_2^2 / (2\sigma^2)}$, with $\sigma > 0$
- Laplace kernel: $K(\mathbf{x}, \mathbf{x}') = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|_1}$, with $\gamma > 0$
- Matérn Kernel: $K(\mathbf{x}, \mathbf{x}') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|_2}{\sigma} \right)^{\nu} K_{\nu} \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|_2}{\sigma} \right)$, where $\sigma > 0$ is the scale parameter, $\nu > 0$ is the smoothness parameter, and K_{ν} is a modified Bessel function of the second kind of order ν .

These kernel functions are continuous and shift-invariant because it can be written as $K(\mathbf{x}, \mathbf{x}') = k(\mathbf{x} - \mathbf{x}')$ for some continuous function $k : \mathcal{X} \rightarrow \mathbb{R}$. As a converse to the previous construction of reproducing kernel from a RKHS, the well-known Moore-Aronszajn theorem states that every symmetric, positive definite kernel defines a unique RKHS.

Consider the regression setting where an unknown function $u(\mathbf{x})$ is accessible via a finite set of samples, i.e. $\{(\mathbf{x}_i, y_i)\}_{i \in [m]}$, learning function $u(\mathbf{x})$ given data samples in $\mathcal{H}_K$ solves the following optimal recovery problem

$$(3.5) \quad \min_{u^* \in \mathcal{H}_K} \|u^*\|_{\mathcal{H}_K} \quad \text{subject to } u^*(\mathbf{x}_i) = y_i \text{ for all } i \in [m].$$

Denote $\mathbf{K} \in \mathbb{R}^{m \times m}$ the kernel matrix with $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ and let $\mathbf{y} = [y_1, \dots, y_m] \in \mathbb{R}^m$. Following the representer theorem [36], the optimization problem has a closed-form solution

$$(3.6) \quad u^*(\mathbf{x}) = \sum_{i=1}^m c_i K(\mathbf{x}, \mathbf{x}_i),$$

where coefficient vector $\mathbf{c} = [c_1, \dots, c_m] \in \mathbb{R}^m$ is computed by $\mathbf{c} = \mathbf{K}^{-1}\mathbf{y}$. However, inverting kernel matrix $\mathbf{K}$ requires $\mathcal{O}(m^3)$ training time, which makes computation infeasible when the sample size m is extremely large.

3.2. Random Features. Random features method is one of the popular techniques to reduce computational complexity of kernel method. The Bochner's theorem [4] from harmonic analysis states that the Fourier transform of continuous shift-invariant kernel (scaled properly) is a proper probability distribution, denoted by $\rho(\boldsymbol{\omega})$. Table 1 shows aforementioned popular kernels and their corresponding Fourier densities Motivated by the Bochner's theorem, we

Kernel name	$k(\mathbf{x}, \mathbf{x}')$	$\rho(\boldsymbol{\omega})$
Gaussian	$e^{-\ \mathbf{x}-\mathbf{x}'\ _2^2/(2\sigma^2)}$	$(2\pi)^{-d/2} \sigma^{2d} \exp(-\frac{\sigma^4 \ \boldsymbol{\omega}\ _2^2}{2})$
Laplace	$e^{-\gamma \ \mathbf{x}-\mathbf{x}'\ _1}$	$(\frac{2}{\pi})^d \prod_{j=1}^d \frac{\gamma}{\gamma^2 + \omega_j^2}$
Matérn	$\frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} \ \mathbf{x}-\mathbf{x}'\ _2}{\sigma} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} \ \mathbf{x}-\mathbf{x}'\ _2}{\sigma} \right)$	$\frac{2^{d+\nu} \pi^{\frac{d}{2}} \Gamma(\nu+d/2) \nu^\nu}{\Gamma(\nu) \sigma^{2\nu}} \left(\frac{2\nu}{\sigma^2} + 4\pi^2 \ \boldsymbol{\omega}\ _2^2 \right)^{-(\nu+d/2)}$

Table 1

Common examples of reproducing kernels and the corresponding Fourier densities. Gaussian, Laplace, and Matérn kernels correspond to Gaussian, Cauchy, and student t distributions, respectively.

have the following Monte Carlo approximation of kernel functions

$$(3.7) \quad K(\mathbf{x}, \mathbf{x}') = \int_{\mathbb{R}^d} \exp(i\langle \boldsymbol{\omega}, \mathbf{x} - \mathbf{x}' \rangle) d\rho(\boldsymbol{\omega}) \approx \frac{1}{N} \sum_{k=1}^N \exp(i\langle \boldsymbol{\omega}_k, \mathbf{x} - \mathbf{x}' \rangle) = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle =: \hat{K}(\mathbf{x}, \mathbf{x}'),$$

where random features $\{\boldsymbol{\omega}_k\}_{k \in [N]}$ are drawn independently from probability distribution $\rho(\boldsymbol{\omega})$ and random Fourier feature map $\phi(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{C}^N$ is defined as ¹

$$(3.8) \quad \phi(\mathbf{x}) = \frac{1}{\sqrt{N}} \left[\exp(i\langle \boldsymbol{\omega}_1, \mathbf{x} \rangle), \dots, \exp(i\langle \boldsymbol{\omega}_N, \mathbf{x} \rangle) \right]^\top.$$

To obtain a real-valued random feature map ϕ , we can use random cosine features. Precisely, we randomly generate i.i.d samples $\{\boldsymbol{\omega}_k\}_{k \in [N]}$ from $\rho(\boldsymbol{\omega})$ and i.i.d samples $\{b_k\}_{k \in [N]}$ from uniform distribution on $[-\pi, \pi]$. We define random cosine feature map $\phi(\tilde{\mathbf{x}}) : \mathbb{R}^{d+1} \rightarrow \mathbb{R}^N$

$$(3.9) \quad \phi(\tilde{\mathbf{x}}) = \frac{1}{\sqrt{N}} \left[\cos(\langle \widetilde{\boldsymbol{\omega}}_1, \tilde{\mathbf{x}} \rangle), \dots, \cos(\langle \widetilde{\boldsymbol{\omega}}_N, \tilde{\mathbf{x}} \rangle) \right]^\top,$$

¹The random feature map depends on random samples $\{\boldsymbol{\omega}_k\}_{k \in [N]}$. Here, we omit this dependency in order to simplify the notation.

where we use notations

$$(3.10) \quad \widetilde{\boldsymbol{\omega}}_k = [\boldsymbol{\omega}_k; b_k] \in \mathbb{R}^{d+1}, \quad \text{and} \quad \widetilde{\mathbf{x}}_k = [\mathbf{x}_k; 1] \in \mathbb{R}^{d+1}$$

to make the inner product in feature maps consistent. In fact, any continuous shift-invariant kernel can be represented using random cosine features [33, 34]. The original shift-invariant kernel function K is approximated by a finite-dimensional kernel function $\hat{K}$. We denote $\mathcal{H}_{\hat{K}}$ the unique RKHS of kernel $\hat{K}$.

Now, we consider the random feature regression problem where we learn function u in $\mathcal{H}_{\hat{K}}$ from training samples $\{(x_i, y_i)\}_{i \in [m]}$. We first draw N i.i.d random features $\{\boldsymbol{\omega}_k\}_{k \in [N]}$ from probability distribution $\rho(\boldsymbol{\omega})$, and then consider the random features model taking form

$$(3.11) \quad u^\sharp(\mathbf{x}) = \sum_{k=1}^N c_k^\sharp \varphi(\mathbf{x}, \boldsymbol{\omega}_k),$$

where $\mathbf{c}^\sharp = [c_1^\sharp, \dots, c_N^\sharp]^T \in \mathbb{R}^N$ are trainable parameters. The random features model is indeed a linear combination of a collection of functions parametrized by weight vector $\boldsymbol{\omega}$, which can be used to construct a sufficiently rich family of functions. Some popular choices for φ are

1. Random Fourier features: $\varphi(\mathbf{x}, \boldsymbol{\omega}) = \exp(i\langle \mathbf{x}, \boldsymbol{\omega} \rangle)$.
2. Random Trigonometric features: $\varphi(\mathbf{x}, \boldsymbol{\omega}) = \cos(\langle \mathbf{x}, \boldsymbol{\omega} \rangle)$ and $\varphi(\mathbf{x}, \boldsymbol{\omega}) = \sin(\langle \mathbf{x}, \boldsymbol{\omega} \rangle)$.
3. Random rectified linear unit (RELU) features: $\varphi(\mathbf{x}, \boldsymbol{\omega}) = \max(\langle \mathbf{x}, \boldsymbol{\omega} \rangle, 0)$.

Denote $\mathbf{A} \in \mathbb{R}^{m \times N}$ the random features matrix defined component-wise by $\mathbf{A}_{j,k} = \varphi(\mathbf{x}_j, \boldsymbol{\omega}_k)$ for $j \in [m]$ and $k \in [N]$. Training the random features model is equivalent to finding the coefficient vector $\mathbf{c}^\sharp$ such that $\mathbf{A}\mathbf{c}^\sharp \approx \mathbf{y}$.

In the under-parameterized regime where we have more measurements than features ($m \geq N$), the coefficients are trained by solving a (regularized) least squares problem:

$$(3.12) \quad \mathbf{c}_\lambda^\sharp \in \underset{\mathbf{c} \in \mathbb{R}^N}{\operatorname{argmin}} \|\mathbf{A}\mathbf{c} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{c}\|_2^2,$$

where $\lambda > 0$ is a regularization parameter. Utilizing random features allows efficient learning with $\mathcal{O}(mN^2)$ time and $\mathcal{O}(mN)$ storage capacity. Therefore, the computational complexity is reduced dramatically in the under-parameterized regime, especially when $N \ll m$.

In recent years, many current studies, especially in the field of Deep learning, emphasize the advantages of over-parametrized models (overparametrization scheme) since those trained models not only fit the training samples exactly but also predict well on unseen test data. In the over-parametrized regime where we have more features than measurements ($N \geq m$), we compute the coefficient vector $\mathbf{c}^\sharp \in \mathbb{R}^N$ by solving the min-norm interpolation problem:

$$(3.13) \quad \mathbf{c}^\sharp \in \underset{\mathbf{c} \in \mathbb{R}^N}{\operatorname{argmin}} \|\mathbf{c}\|_2^2 \quad \text{subject to } \mathbf{A}\mathbf{c} = \mathbf{y}.$$

The solution $\mathbf{c}^\sharp$ can be viewed as the limit of $\mathbf{c}_\lambda^\sharp$ as $\lambda \rightarrow 0$.

While random features method was originally proposed to approximate large-scale kernel machines, it has been used to study neural networks [16, 44, 31] since it is indeed a linearized neural network. Precisely, the random features model is a two-layer (one hidden layer) neural

network that only the weights of the output layer are trainable. The weights connecting input layer and hidden layer are randomly sampled from a known distribution. Unlike solving non-convex optimization problems to train neural networks, the training of random feature models leads to solving a convex optimization problem.

4. Computational Framework. In this section, we introduce a random features method to solve the linear PDE (2.10), and hence approximate the Koopman eigenfunctions. Then, we describe how we use Koopman eigenfunctions to approximate Lyapunov functions.

4.1. Solving linear PDE using random features. Recall that the Koopman eigenfunction can be decomposed into linear and nonlinear parts, which satisfy conditions in (2.10). The linear part can be found by finding the left eigenvector of Jacobian matrix $\mathbf{E} := \frac{\partial f}{\partial \mathbf{x}}(0)$. Then, the problem of approximating Koopman eigenfunction reduces to approximating its nonlinear part. Our approach is to use random features model $h^\sharp(\mathbf{x})$ in the form of (3.11) to approximate the nonlinear part by solving the linear PDE. Given a set of collocation points $\{\mathbf{x}_j\}_{j \in [m]} \subset \mathbb{R}^d$, the linear PDE is satisfied by enforcing the PDE constraint at collocation points. Precisely, for each eigenvalue λ , the random features approximation $h^\sharp(\mathbf{x})$ satisfies

$$\begin{aligned} h^\sharp(0) &= 0 \\ \nabla h^\sharp(0) &= 0 \\ \nabla h^\sharp(\mathbf{x}_j) \cdot f(\mathbf{x}_j) - \lambda h^\sharp(\mathbf{x}_j) + \mathbf{w}^\top G(\mathbf{x}_j) &= 0, \quad \text{for all } j \in [m]. \end{aligned} \quad (4.1)$$

Since random features model is linear in terms of coefficient vector $\mathbf{c}^\sharp$ and the PDE is linear as well, then we can train the random features model by solving a linear system. Define a stacked matrix $\mathbf{A} \in \mathbb{R}^{(1+d+m) \times N}$ of form

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}^{(1)} \in \mathbb{R}^{1 \times N} \\ \mathbf{A}^{(2)} \in \mathbb{R}^{d \times N} \\ \mathbf{A}^{(3)} \in \mathbb{R}^{m \times N} \end{bmatrix}. \quad (4.2)$$

Matrix $\mathbf{A}^{(1)}$ corresponds to condition $h^\sharp(0) = 0$, and hence has entries $\mathbf{A}_k^{(1)} = \varphi(0, \boldsymbol{\omega}_k)$. Matrix $\mathbf{A}^{(2)}$ is associated to condition $\nabla h^\sharp(0) = 0$ and its k -th column is $\nabla \varphi(0, \boldsymbol{\omega}_k)$. Matrix $\mathbf{A}^{(3)}$ is derived from linear PDE at collocation points and has entries $\mathbf{A}_{j,k}^{(3)} = \nabla \varphi(\mathbf{x}_j, \boldsymbol{\omega}_k) \cdot f(\mathbf{x}_j) - \lambda \varphi(\mathbf{x}_j, \boldsymbol{\omega}_k)$. Define vector $\mathbf{y} \in \mathbb{R}^{1+d+m}$ with entries

$$\mathbf{y}_j = \begin{cases} 0, & \text{if } j = 1, \dots, d+1 \\ -\mathbf{w}^\top G(\mathbf{x}_{j-d-1}), & \text{if } j = d+2, \dots, d+m+1 \end{cases} \quad (4.3)$$

We can rewrite constraints in (4.1) as a linear system $\mathbf{A}\mathbf{c}^\sharp = \mathbf{y}$. If the linear system is over-determined, i.e. $1+d+m > N$, we consider the least-squares solution. The min-norm solution will be adopted when the linear system is under-determined ($1+d+m \leq N$). Once we obtain the approximated nonlinear part $h^\sharp(\mathbf{x})$, then we can approximate principal eigenfunction by $\psi_\lambda(\mathbf{x}) = \mathbf{w}_\lambda^\top \mathbf{x} + h^\sharp(\mathbf{x})$, where λ is the eigenpair of $\mathbf{E}$.

4.2. Approximating Lyapunov functions using Koopman eigenfunctions.

Lyapunov functions are scalar-valued functions $V : \mathbb{R}^d \rightarrow \mathbb{R}_+$ that provide a rigorous framework for equilibrium stability analysis in dynamical systems without explicitly solving the underlying equations. A function $V(\mathbf{x})$ qualifies as a Lyapunov function if it satisfies

- Positivity: $V(\mathbf{x}) > 0$ for all $x \neq 0$ and $V(0) = 0$;
- Non-increasing behavior: Define $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot f(\mathbf{x})$, then $\dot{V}(\mathbf{x}) \leq 0$ along trajectories of the system.

A non-constructive result on the existence of Lyapunov functions was established by Massera [26], which motivates the numerical method on the construction of Lyapunov functions. Algorithms based on radial basis functions and PDEs are widely used. An approach based on Koopman eigenfunctions was proposed in [23]. It suggests the following ansatz:

$$(4.4) \quad V(\mathbf{x}) = \sum_{i,j=1}^n P_{ij} \psi_i(\mathbf{x}) \psi_j(\mathbf{x}),$$

where $\psi_i(\mathbf{x})$ are the principal eigenfunctions and the matrix $\mathbf{P}$ defined component-wise by $\mathbf{P}_{i,j} = P_{ij}$ satisfies

$$(4.5) \quad \mathbf{E}^\top \mathbf{P} + \mathbf{P} \mathbf{E} \prec 0.$$

In practice, the semidefinite program can indeed be solved in Python using CVXPY [13]. The true principal eigenfunctions are not always known to user. Therefore, our approach is to replace true eigenfunctions $\psi_i(\mathbf{x})$ by their approximation derived by random features method in previous section. Denote $\hat{\psi}_i(\mathbf{x})$ the approximated Koopman eigenfunctions using random features, the approximated Lyapunov function $\hat{V}(\mathbf{x})$ is defined as

$$(4.6) \quad \hat{V}(\mathbf{x}) = \sum_{i,j=1}^n P_{ij} \hat{\psi}_i(\mathbf{x}) \hat{\psi}_j(\mathbf{x}),$$

where the coefficients $P_{i,j}$ are computed by solving semidefinite program (4.5).

5. Error Estimates. In this section, we provide an error estimate for our method. The roadmap for our proof is first analyzing the kernel-based method and then consider the error analysis of approximating kernel method by using random features method.

Assumption 5.1. The following assumptions hold:

- **(A1)** Let the equilibrium point $\mathbf{x}_e$ of the dynamical system (2.1) be hyperbolic. Assume that $f \in C^s(D, \mathbb{R}^d)$ for some integer $s \geq 1$, where D is an open neighborhood around $\mathbf{x}_e$ with finite volume, i.e. $\text{vol}(D) < \infty$.
- **(A2)** Assume that the solution of linear PDE (2.10) belongs to some RKHS $\mathcal{H}_K$, which is continuously embedded in Sobolev space $H^{s+1}(D)$.
- **(A3)** Let $Z \subset D$ be a set of collocation points with fill-in distance

$$(5.1) \quad \rho_{Z,D} = \sup_{\mathbf{x} \in D} \min_{\mathbf{z} \in Z} \|\mathbf{x} - \mathbf{z}\|_2.$$

Remark 5.2. Assumption **(A1)** addresses the smoothness of dynamical system. Assumption **(A2)** is a standard assumption when we analyze kernel-based PDE solvers. It is satisfied whenever kernel function K has smoothness $C^{s+1}(D)$ in both variables separately and $D \subset \mathbb{R}^d$ has a sufficiently smooth boundary. Some popular kernels satisfy the condition, such as Gaussian kernel, Laplace kernel, and Matérn kernel of suitable smoothness. Moreover, $H^s(D)$ is indeed a RKHS when $s \geq d/2$. Collocation points in Assumption **(A3)** are used to solve the linear PDE in (2.10).

The main result of this section is presented in the next theorem

Theorem 5.3. Suppose that Assumption 5.1 holds with some constant $s \geq d/2$. Let h_λ be the solution of linear PDE in (2.10) corresponding to eigenvalue λ . Then there exists a finite sum random features model $h_\lambda^\sharp$ taking form (3.11) and there are some constants ρ_0 and $C > 0$ depending on s, d, D such that if $\rho_{Z,D} < \rho_0$ it holds with probability at least $1 - \delta$ over drawing random features

$$(5.2) \quad \|h_\lambda - h_\lambda^\sharp\|_{L^2(D)} \leq C (\|f\|_{H^s(D)} + |\lambda|) \rho_{Z,D}^s + \left(1 + \frac{12 \log(2/\delta) \text{vol}(D)}{\sqrt{N}}\right) \|h_\lambda\|_{\mathcal{H}_K}.$$

Remark 5.4. The first term in the error bound comes from solving linear PDE with kernel-based method. In the second term, the decay rate $1/\sqrt{N}$ comes from Monte Carlo sampling when we approximate kernel functions using random features method.

The next theorem addresses the error estimates of kernel-based method.

Theorem 5.5. ([22, Theorem 5]) Suppose that Assumption 5.1 holds with some constant $s \geq d/2$. Let h_λ be the solution of linear PDE in (2.10) corresponding to eigenvalue λ . Then there exists $h_\lambda^* \in \mathcal{H}_K$ and there are some constants ρ_0 and $C > 0$ depending on s, d, D such that if $\rho_{Z,D} < \rho_0$ it holds

$$(5.3) \quad \|h_\lambda - h_\lambda^*\|_{L^2(D)} \leq C (\|f\|_{H^s(D)} + |\lambda|) \rho_{Z,D}^s + \|h_\lambda\|_{\mathcal{H}_K}.$$

Our next step is to approximate functions in $\mathcal{H}_K$ using random features model. We adopt an alternative representation of preselected RKHS $\mathcal{H}_K$. Precisely, we define the following function space

$$(5.4) \quad \mathcal{F}(\rho) := \left\{ f(\mathbf{x}) = \int_{\mathbb{R}^d} \alpha(\boldsymbol{\omega}) \varphi(\mathbf{x}, \boldsymbol{\omega}) d\rho(\boldsymbol{\omega}) : \|f\|_\rho^2 = \mathbb{E}_{\boldsymbol{\omega}}[\alpha(\boldsymbol{\omega})^2] < \infty \right\},$$

where $\rho(\cdot)$ is the Fourier transform density associated with kernel K . Notice that the completion of $\mathcal{F}(\rho)$ is a Hilbert space equipped with RKHS norm $\|f\|_\rho$. Recall the Proposition 4.1 in [34], it is indeed the reproducing kernel Hilbert space $\mathcal{H}_K$. The endowed norms $\|\cdot\|_{\mathcal{H}_K}$ and $\|\cdot\|_\rho$ are equivalent. The next theorem addresses the approximation ability of finite sum random features model.

Theorem 5.6. ([24, Theorem 3]) Let h be a function from $\mathcal{F}(\rho)$. Suppose that the random feature map ϕ satisfies $|\varphi(\mathbf{x}, \boldsymbol{\omega})| \leq 1$ for all $\mathbf{x} \in X$ and $\boldsymbol{\omega} \in \mathbb{R}^d$. Then for any $\delta \in (0, 1)$, there exists $c_1^\sharp, \dots, c_N^\sharp$ so that the function

$$(5.5) \quad f^\sharp(\mathbf{x}) = \sum_{k=1}^N c_k^\sharp \phi(\mathbf{x}, \boldsymbol{\omega}_k)$$

satisfies

$$\|h(\mathbf{x}) - h^\sharp(\mathbf{x})\|_{L^2(D)} \leq \frac{12\|h\|_\rho \log(2/\delta) \text{vol}(D)}{\sqrt{N}}$$

with probability at least $1 - \delta$ over $\omega_1, \dots, \omega_N$ drawn i.i.d from $\rho(\omega)$.

With the above two theorems at hand, the main result of this subsection follows easily.

Proof of Theorem 5.3. Let h_λ be the solution of PDE corresponding to eigenvalue λ . Using Theorem 5.5 yields a function $h_\lambda^* \in \mathcal{H}_K$ such that h_λ is well-approximated by h_λ^* . Then, we can construct a random features model $h_\lambda^\sharp$ to approximate h_λ^* using Theorem 5.6. Then we have

$$\begin{aligned} \|h_\lambda - h_\lambda^\sharp\|_{L^2(D)} &\leq \|h_\lambda - h_\lambda^*\|_{L^2(D)} + \|h_\lambda^* - h_\lambda^\sharp\|_{L^2(D)} \\ &\leq C(\|f\|_{H^s(D)} + |\lambda|) \rho_{Z,D}^s + \|h_\lambda\|_{\mathcal{H}_K} + \frac{12\|h_\lambda^*\|_\rho \log(2/\delta) \text{vol}(D)}{\sqrt{N}} \end{aligned}$$

The desired bound follows from the facts that the endowed norms $\|\cdot\|_{\mathcal{H}_K}$ and $\|\cdot\|_\rho$ are equivalent and the construction of h_λ^* satisfying $\|h_\lambda^*\|_{\mathcal{H}_K} \leq \|h_\lambda\|_{\mathcal{H}_K}$. $\blacksquare$

6. Numerical Experiments. In this section, we perform some numerical experiments on testing the performance of our proposed method using examples from [12, 23, 22]. Furthermore, we compare our method with kernel-based method in terms of test accuracy and training time. We consider Gaussian random features in all examples and the choices of variance σ^2 will be clearly stated for each example. We use numpy.pinv method to solve the linear system. The training time is recorded using Python time library. The reproducible can be found in an open repository: <https://github.com/liaochunyang/Koopman>.

6.1. Analytic Example 1. We consider the following dynamical system

$$\dot{\mathbf{x}} = \begin{bmatrix} -2x_1 \\ -3(x_2 - x_1^2) \end{bmatrix}.$$

We can check that there are eigenvalues $\lambda_1 = -2$ and $\lambda_2 = -3$. The corresponding eigenfunctions are $\psi_{\lambda_1}(\mathbf{x}) = x_1$ and $\psi_{\lambda_2}(\mathbf{x}) = x_2 + 3x_1^2$. We generate 900 grid points over squared domain $[-5, 5] \times [-5, 5]$ to solve the linear PDE. We then uniformly generate 3600 points to test model performance. We use $N = 100$ Gaussian random features with variance $\sigma^2 = 0.2$ to construct the random features model. In Figures 1 and 2, we show the contour plots of learned and true eigenfunctions. We observe that the principal eigenfunctions are accurately estimated.

After learning the principal eigenfunctions, we then follow the construction in Section 4.2 to approximate the Lyapunov function. The approximated Lyapunov function, true Lyapunov functions, the approximated and true $\dot{V}(\mathbf{x})$ are depicted in Figure 3. We observe that the Lyapunov function can be approximated accurately using our method. Moreover, the numerical results show that the positivity and non-increasing behavior of the Lyapunov functions.

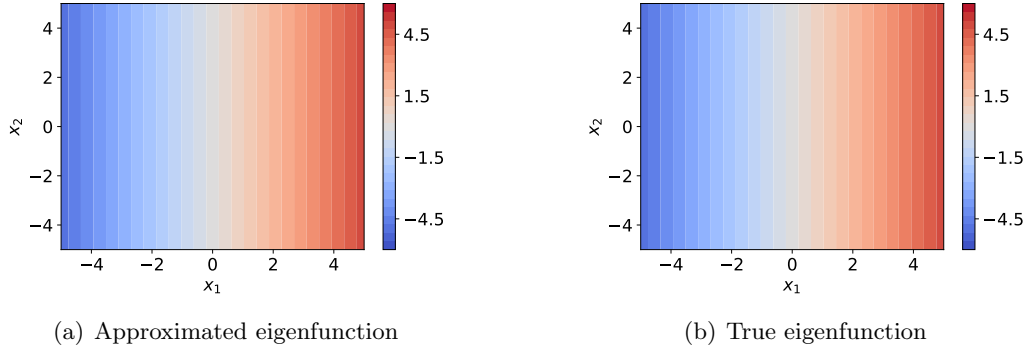


Figure 1. Approximated eigenfunction using random features model (left) and true eigenfunction ψ_{λ_1} (right).

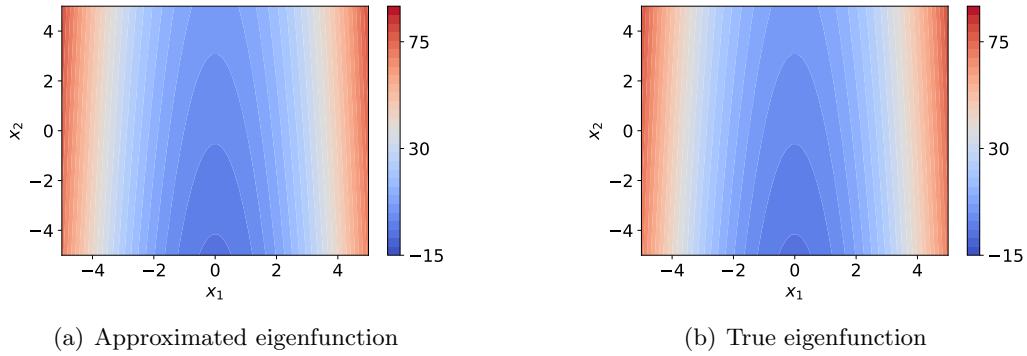


Figure 2. Approximated eigenfunction using random features model (left) and true eigenfunction ψ_{λ_2} (right).

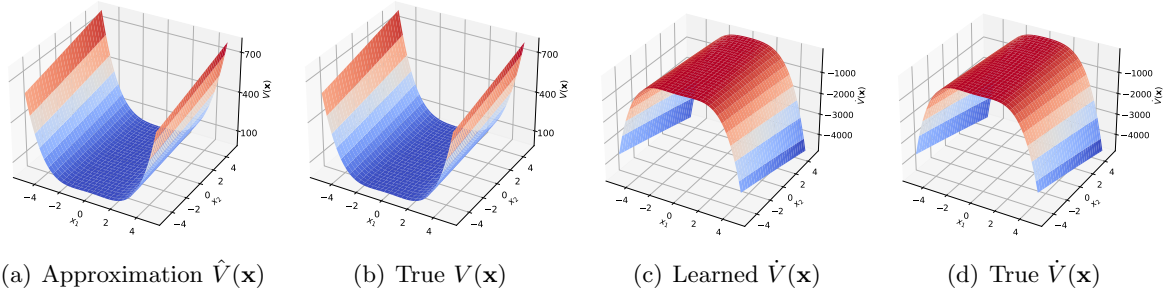


Figure 3. We show approximated Lyapunov function, true Lyapunov function, the approximated $\dot{V}(\mathbf{x})$ and true $\dot{V}(\mathbf{x})$.

6.2. Analytic Example 2. The second analytic example was considered in [12, 22]. The two dimensional dynamical system is given by

$$\dot{\mathbf{x}} = \begin{bmatrix} -2\lambda_2 x_2 (x_1^2 - x_2 - 2x_1 x_2 + x_2^4) + \lambda_1 (x_1 + 4x_1^2 x_2 - x_2^2 - 8x_1 x_2^3 + 4x_2^5) \\ 2\lambda_1 (x_1 - x_2^2)^2 - \lambda_2 (x_1^2 - x_2 - 2x_1 x_2 + x_2^4) \end{bmatrix},$$

where λ_1 and λ_2 are the eigenvalues of the system when it is linearized at the origin. In this example, we pick $\lambda_1 = -1$ and $\lambda_2 = 3$ such that the system has a saddle equilibrium point at the origin. The eigenfunctions correspond to principal eigenvalues are $\psi_{\lambda_1} = x_1 - x_2^2$ and $\psi_{\lambda_2} = -x_1^2 + x_2 + 2x_1x_2^2 - x_2^4$.

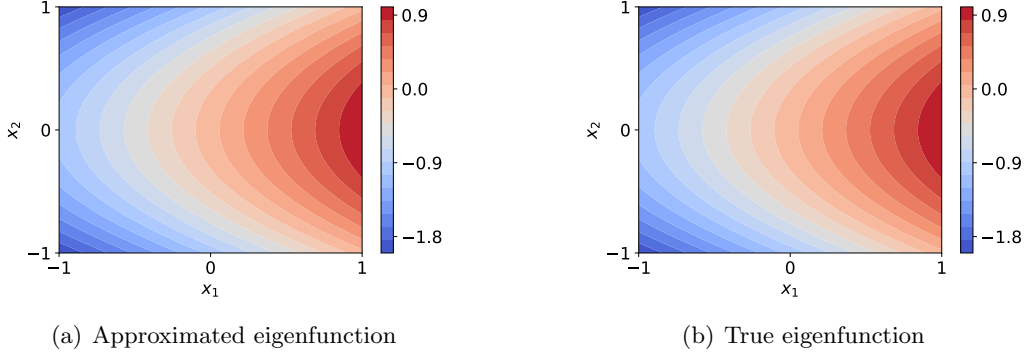


Figure 4. *Approximated eigenfunction using random features model (left) and true eigenfunction ψ_{λ_1} (right).*

We take 900 grid points as collocation points over squared domain $[-1, 1] \times [-1, 1]$ to solve the linear PDE. In order to measure the model performance, we take 3600 grid points over the same domain as test points. For eigenfunction ψ_1 , we randomly sample $N = 100$ i.i.d Gaussian random features with variance $\sigma^2 = 0.02$. The learned eigenfunction using random features method and the analytical eigenfunction are shown in Figure 4. For the unstable eigenvalue $\lambda_2 = 3$, we also use $N = 100$ i.i.d Gaussian random features with variance $\sigma^2 = 0.02$ and depict the learned and true eigenfunctions in Figure 5.

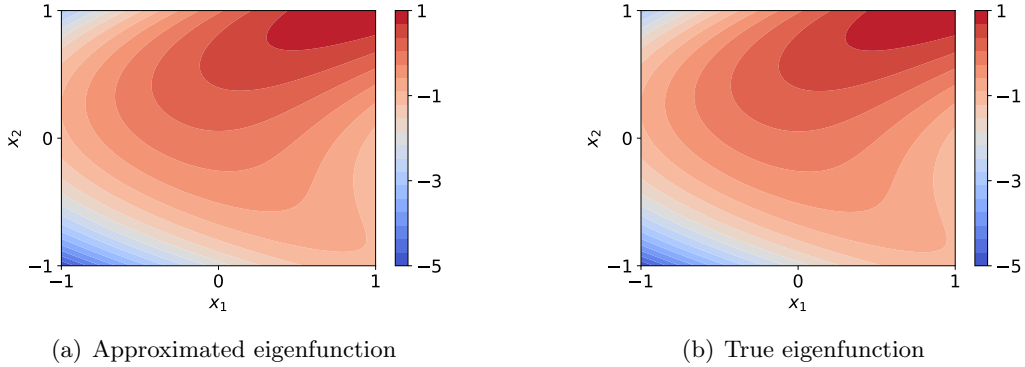


Figure 5. *Approximated eigenfunction using random features model (left) and true eigenfunction ψ_{λ_2} (right).*

Finally, we use learned eigenfunctions to approximate the Lyapunov function following the construction in Section 4.2. In Figure 6, we show the learned and true Lyapunov functions and further show learned and true $\dot{V}(\mathbf{x})$. Our results indicate that the principal eigenfunctions and the Lyapunov function can be approximated accurately using our proposed random features method.

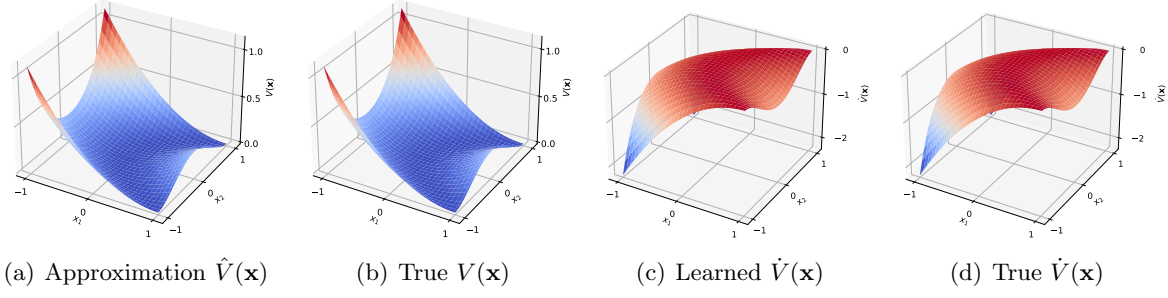


Figure 6. We show approximated Lyapunov function, true Lyapunov function, the approximated $\dot{V}(\mathbf{x})$ and true $\dot{V}(\mathbf{x})$.

6.3. Example 3.

The third analytic example is taken from [22] as well. We consider the dynamical system

$$\dot{\mathbf{x}} = \begin{bmatrix} \frac{(7.5x_2^2+5)(x_1^3+x_1+\sin(x_2))+(-x_1+x_2^3+2x_2)\cos(x_2)}{9x_1^2x_2^2+6x_1^2+3x_2^2+\cos(x_2)+2} \\ \frac{2.5x_1^3+2.5x_1-(3x_1^2+1)(-x_1+x_2^3+2x_2)+2.5\sin(x_2)}{9x_1^2x_2^2+6x_1^2+3x_2^2+\cos(x_2)+2} \end{bmatrix}.$$

The vector field for this system is smooth everywhere on $\mathbb{R}^2$. The system has a saddle point at origin. The eigenpairs of this system are

$$\lambda_1 = 2.5, \psi_{\lambda_1}(\mathbf{x}) = x_1 + \sin(x_2) + x_1^3, \text{ and } \lambda_2 = -1, \psi_{\lambda_2}(\mathbf{x}) = x_1 - 2x_2 - x_2^3.$$

We use 900 grid points over squared domain $[1.5, 2.5] \times [1.5, 2.5]$ to solve the linear PDE. Test dataset contains grid points of size 3600. For each eigenfunction, we randomly generate $N = 100$ i.i.d Gaussian random features with variance $\sigma^2 = 0.02$. In Figure 7, we show the learned eigenfunction using random features model, true eigenfunction ψ_{λ_1} , and absolute pointwise error. For eigenvalue $\lambda_2 = -1$, we show the learned eigenfunction, true eigenfunction ψ_{λ_2} , and absolute pointwise error in Figure 8.

6.4. The Duffing oscillator.

We consider the unforced Duffing oscillator

$$\dot{\mathbf{x}} = \begin{bmatrix} x_2 \\ x_1 - \delta x_2 - x_1^3 \end{bmatrix},$$

where we use $\delta = 0.5$ for eigenfunction computation. The dynamical system has one unstable equilibrium point at the origin, and two stable equilibrium points at $(-1, 0)$ and $(0, 1)$. In this example, we use 3600 grid points over the domain $[-2, 2] \times [-2, 2]$ as collocation points to solve the linear PDE. For the random features model, we randomly sample $N = 1000$ weights from Gaussian distribution with variance $\sigma^2 = 2 \times 10^{-6}$. We only aim to learn the eigenfunction corresponding to eigenvalue $\lambda = \frac{\sqrt{17}-1}{4} \approx 0.78$. In Figure 9, we depict our learned eigenfunction and the stable manifold which is the zero-level set (black curve) of

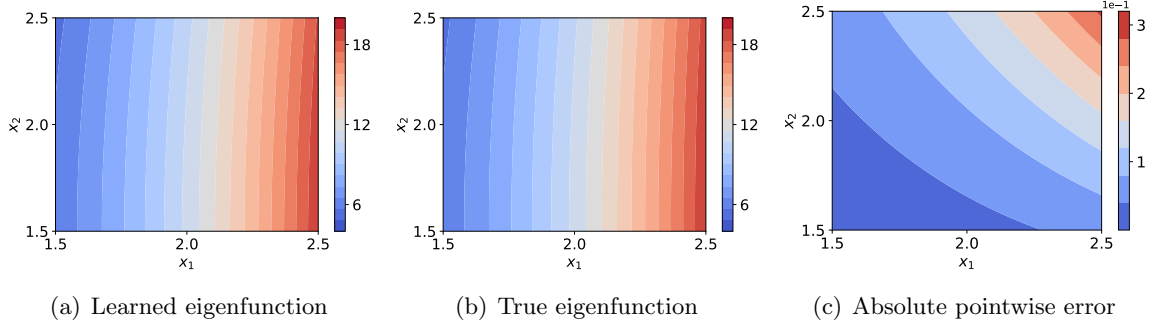


Figure 7. Eigenvalue $\lambda_1 = 2.5$: learned eigenfunction using random features model (left), true eigenfunction (middle), and absolute pointwise error (right).

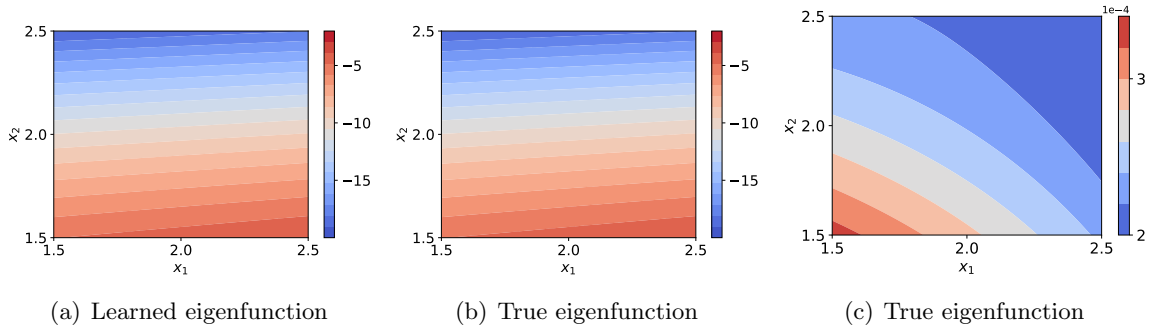


Figure 8. Eigenvalue $\lambda_2 = -1$: learned eigenfunction using random features model (left), true eigenfunction (middle), and absolute pointwise error (right).

415 learned eigenfunction. We observe that our approach accurately captures the behavior of the
Koopman eigenfunction up to scaling.

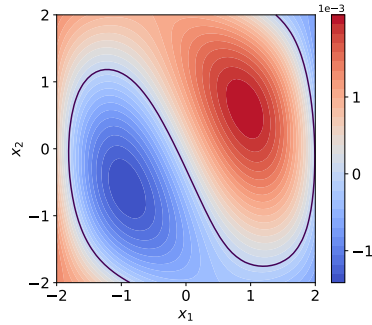


Figure 9. Learned eigenfunction using random features model corresponds to $\lambda = \frac{\sqrt{17}-1}{4}$ and its zero-level set.

416

417 **6.5. Comparison.** In this section, we compare our proposed random features method with
418 kernel-based method. We consider the analytic examples 1-3 since the true eigenfunctions are

known to users. Table 2 summarizes the errors and training time of each method. We refer RF to our proposed random features method. Let X_{test} be the test samples and denote $|X_{test}|$ its size. Let $\psi(\mathbf{x})$ be the true eigenfunction and $\hat{\psi}$ be its approximation, we define test average error (Ave error) and test relative error (Rel error) as

$$(6.1) \quad \begin{aligned} Ave\ error &= \frac{1}{|X_{test}|} \sum_{\mathbf{x}_j \in X_{test}} \left(\psi(\mathbf{x}_j) - \hat{\psi}(\mathbf{x}_j) \right)^2, \\ Rel\ error &= \frac{\sum_{\mathbf{x}_j \in X_{test}} \left(\psi(\mathbf{x}_j) - \hat{\psi}(\mathbf{x}_j) \right)^2}{\sum_{\mathbf{x}_j \in X_{test}} \psi(\mathbf{x}_j)^2}. \end{aligned}$$

The code to kernel-based method can be found in a public repository². To make the comparison fair, we modify the code by using Numpy library. In all examples, we use 900 grid points over the domain to train the models and take another 3600 grid points over the same domain to test model performance (as mentioned in previous subsections). Overall, we observed that the performance of our proposed method is stable cross all analytic examples suggesting that our method is reliable and provides a good baseline for approximating Koopman eigenfunctions. Moreover, our proposed random features method achieves similar error or even beats kernel-based method. Furthermore, our method has less computational complexity in terms of training time. The implementation of kernel-based method usually requires high-performance numerical computing library, such as JAX, especially when the sample size is large. Our proposed method is easy to implement and can be done without using JAX.

Example	Method	Eigenfunction ψ_{λ_1}		Eigenfunction ψ_{λ_2}		Time (seconds)
		Ave error	Rel error	Ave error	Rel error	
Analytic Example 1	RF	0	0	3.96×10^{-7}	3.27×10^{-10}	0.03
	Kernel	0	0	1.28×10^3	9.93×10^{-1}	43.60
Analytic Example 2	RF	3.19×10^{-5}	5.72×10^{-5}	9.91×10^{-6}	8.79×10^{-6}	0.02
	Kernel	1.22×10^{-7}	2.08×10^{-7}	8.60×10^{-1}	7.07×10^{-1}	42.00
Analytic Example 3	RF	1.24×10^{-2}	8.56×10^{-5}	3.92×10^{-7}	3.06×10^{-9}	0.02
	Kernel	1.54×10^{-5}	1.21×10^{-5}	6.41	2.24	38.43

Table 2

Summary of numerical results: we report the average and relative test errors of our numerical experiments and compare our approach with kernel method. We use $N = 100$ random features here.

As our theory suggests, increasing the number of features leads to better performance. To numerically verify this, we rethink analytic examples 2 and 3. For analytic example 2, we use $N = 500$ random features. In analytic example 3, we use $N = 700$ random features. We summarize our results in Table 3. We observe that the test performance of the random features method can be improved when we increase the number of features. It is remarkable that the random features method has better performance than kernel method and the training is much faster than kernel method. Therefore, our results suggests that our method provides a good baseline.

²<https://github.com/jonghyeon1998/Koopman>

Example	Eigenfunction ψ_{λ_1}		Eigenfunction ψ_{λ_2}		Time (seconds)
	Ave error	Rel error	Ave error	Rel error	
Analytic Example 2	4.89×10^{-8}	8.76×10^{-8}	4.36×10^{-8}	3.86×10^{-8}	0.18
Analytic Example 3	1.35×10^{-3}	9.29×10^{-6}	1.05×10^{-7}	8.20×10^{-10}	0.27

Table 3

Summary of numerical results: we report average and relative test errors and training time of random features method. For analytic example 2, we use $N = 500$ random features. In analytic example 3, we use $N = 700$ random features.

7. Conclusion. In this paper, we consider the approximation of the eigenfunctions of Koopman operator, which has been widely used in understanding the behavior of dynamical systems. We introduced a random features method for approximating the principal eigenfunctions. Our approach is based on the decomposition of Koopman eigenfunctions, which has linear and nonlinear components. It was shown that the linear part corresponds to the local linear dynamics of the system and the nonlinear part satisfies a linear partial differential equation. Our approach solves the linear partial differential equation using random features method. In addition to describing the computational framework, we provide an error estimate for our method. Numerical experiments indicate that our method can accurately capture the behavior of Koopman eigenfunctions. Moreover, our method is easy to implement and reduces the training time significantly. These properties suggest that our proposed method is a good benchmark in learning Koopman eigenfunctions.

REFERENCES

- [1] N. ARONSZAJN, *Theory of reproducing kernels*, Transactions of the American Mathematical Society, 68 (1950), pp. 337–404.
- [2] P. BATLLE, Y. CHEN, B. HOSSEINI, H. OWHADI, AND A. M. STUART, *Error analysis of kernel/GP methods for nonlinear and parametric pdes*, Journal of Computational Physics, 520 (2025), p. 113488, <https://doi.org/https://doi.org/10.1016/j.jcp.2024.113488>, <https://www.sciencedirect.com/science/article/pii/S0021999124007368>.
- [3] P. BATLLE, M. DARCY, B. HOSSEINI, AND H. OWHADI, *Kernel methods are competitive for operator learning*, Journal of Computational Physics, 496 (2024), p. 112549, <https://doi.org/https://doi.org/10.1016/j.jcp.2023.112549>, <https://www.sciencedirect.com/science/article/pii/S0021999123006447>.
- [4] S. BOCHNER, *Harmonic Analysis and the Theory of Probability*, University of California Press Berkeley, 1955.
- [5] S. L. BRUNTON, M. BUDIŠIĆ, E. KAISER, AND J. N. KUTZ, *Modern koopman theory for dynamical systems*, SIAM Review, 64 (2022), pp. 229–340, <https://doi.org/10.1137/21M1401243>, <https://doi.org/10.1137/21M1401243>, <https://arxiv.org/abs/https://doi.org/10.1137/21M1401243>.
- [6] J. CHEN, X. CHI, W. E, AND Z. YANG, *Bridging traditional and machine learning-based algorithms for solving PDEs: The random feature method*, Journal of Machine Learning, 1 (2022), pp. 268–298, <https://doi.org/https://doi.org/10.4208/jml.220726>, http://global-sci.org/intro/article_detail/jml/21029.html.
- [7] Y. CHEN, B. HOSSEINI, H. OWHADI, AND A. M. STUART, *Solving and learning nonlinear pdes with gaussian processes*, Journal of Computational Physics, 447 (2021), p. 110668, <https://doi.org/https://doi.org/10.1016/j.jcp.2021.110668>, <https://www.sciencedirect.com/science/article/pii/S0021999121005635>.
- [8] Z. CHEN AND H. SCHAEFFER, *Conditioning of random Fourier feature matrices: double descent and generalization error*, Information and Inference: A Journal of the IMA, 13 (2024), p. iaad054,

- <https://doi.org/10.1093/imaiai/iaad054>, <https://doi.org/10.1093/imaiai/iaad054>, <https://arxiv.org/abs/https://academic.oup.com/imaiai/article-pdf/13/2/iaad054/57172529/iaad054.pdf>.
- [9] M. J. COLBROOK, I. MEZIĆ, AND A. STEPANENKO, *Limits and powers of Koopman learning*, arXiv: 2407.06312, (2024), <https://arxiv.org/abs/2407.06312>.
- [10] S. DAS AND D. GIANNAKIS, *Koopman spectra in reproducing kernel Hilbert spaces*, Applied and Computational Harmonic Analysis, 49 (2020), pp. 573–607, <https://doi.org/https://doi.org/10.1016/j.acha.2020.05.008>, <https://www.sciencedirect.com/science/article/pii/S1063520320300427>.
- [11] E. B. DAVIES AND M. PLUM, *Spectral pollution*, IMA Journal of Numerical Analysis, 24 (2004), pp. 417–438, <https://doi.org/10.1093/imanum/24.3.417>, <https://doi.org/10.1093/imanum/24.3.417>, <https://arxiv.org/abs/https://academic.oup.com/imaajna/article-pdf/24/3/417/2069407/240417.pdf>.
- [12] S. A. DEKA, S. S. NARAYANAN, AND U. VAIDYA, *Path-integral formula for computing Koopman eigenfunctions*, in 2023 62nd IEEE Conference on Decision and Control (CDC), 2023, pp. 6641–6646, <https://doi.org/10.1109/CDC49753.2023.10384288>.
- [13] S. DIAMOND AND S. BOYD, *CVXPY: A Python-embedded modeling language for convex optimization*, Journal of Machine Learning Research, 17 (2016), pp. 1–5.
- [14] Y. GUO, M. SCHALLER, K. WORTHMAN, AND S. STREIF, *Modularized data-driven approximation of the Koopman operator and generator*, Physica D: Nonlinear Phenomena, 476 (2025), p. 134651, <https://doi.org/https://doi.org/10.1016/j.physd.2025.134651>, <https://www.sciencedirect.com/science/article/pii/S0167278925001307>.
- [15] B. HOU, S. SANJARI, N. DAHLIN, S. BOSE, AND U. VAIDYA, *Sparse learning of dynamical systems in RKHS: An operator-theoretic approach*, in Proceedings of the 40th International Conference on Machine Learning, vol. 202 of Proceedings of Machine Learning Research, PMLR, 23–29 Jul 2023, pp. 13325–13352, <https://proceedings.mlr.press/v202/hou23c.html>.
- [16] A. JACOT, F. GABRIEL, AND C. HONGLER, *Neural tangent kernel: Convergence and generalization in neural networks*, in Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., 2018, https://proceedings.neurips.cc/paper_files/paper/2018/file/5a4be1fa34e62bb8a6ec6b91d2462f5a-Paper.pdf.
- [17] S. KLUS, F. NÜSKE, S. PEITZ, J.-H. NIEMANN, C. CLEMENTI, AND C. SCHÜTTE, *Data-driven approximation of the Koopman generator: Model reduction, system identification, and control*, Physica D: Nonlinear Phenomena, 406 (2020), p. 132416, <https://doi.org/https://doi.org/10.1016/j.physd.2020.132416>, <https://www.sciencedirect.com/science/article/pii/S0167278919306086>.
- [18] B. O. KOOPMAN, *Hamiltonian systems and transformation in Hilbert space*, Proceedings of the National Academy of Sciences, 17 (1931), pp. 315–318, <https://doi.org/10.1073/pnas.17.5.315>, <https://www.pnas.org/doi/abs/10.1073/pnas.17.5.315>, <https://arxiv.org/abs/https://www.pnas.org/doi/pdf/10.1073/pnas.17.5.315>.
- [19] B. O. KOOPMAN AND J. V. NEUMANN, *Dynamical systems of continuous spectra*, Proceedings of the National Academy of Sciences, 18 (1932), pp. 255–263, <https://doi.org/10.1073/pnas.18.3.255>, <https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255>, <https://arxiv.org/abs/https://www.pnas.org/doi/pdf/10.1073/pnas.18.3.255>.
- [20] Y. LAN AND I. MEZIĆ, *Linearization in the large of nonlinear systems and Koopman operator spectrum*, Physica D: Nonlinear Phenomena, 242 (2013), pp. 42–53, <https://doi.org/https://doi.org/10.1016/j.physd.2012.08.017>, <https://www.sciencedirect.com/science/article/pii/S0167278912002102>.
- [21] A. LASOTA AND M. C. MACKEY, *Chaos, Fractals, and Noise: Stochastic Aspects of Dynamics*, Springer New York, NY, 2 ed., 1994.
- [22] J. LEE, B. HAMZI, B. HOU, H. OWHADI, G. SANTIN, AND U. VAIDYA, *Kernel methods for the approximation of the eigenfunctions of the Koopman operator*, Physica D: Nonlinear Phenomena, 476 (2025), p. 134662, <https://doi.org/https://doi.org/10.1016/j.physd.2025.134662>, <https://www.sciencedirect.com/science/article/pii/S0167278925001411>.
- [23] J. LEE, B. HAMZI, H. OWHADI, AND U. VAIDYA, *A note on kernel methods for the construction of Lyapunov functions using Koopman eigenfunctions*, Preprint, (2024).
- [24] C. LIAO, *Solving partial differential equations with random feature models*, arXiv:2501.00288, (2024). Submitted.
- [25] C. LIAO, D. NEEDELL, AND H. SCHAEFFER, *Cauchy random features for operator learning in sobolev space*, arXiv: 2503.00300, (2025), <https://arxiv.org/abs/2503.00300>.

- [26] J. L. MASSERA, *On liapounoff's conditions of stability*, Annals of Mathematics, 50 (1949), pp. 705–721, <http://www.jstor.org/stable/1969558> (accessed 2025-06-17).
- [27] A. MAUROY AND I. MEZIĆ, *Global stability analysis using the eigenfunctions of the Koopman operator*, IEEE Transactions on Automatic Control, 61 (2016), pp. 3356–3369, <https://doi.org/10.1109/TAC.2016.2518918>.
- [28] S. MEI, T. MISIAKIEWICZ, AND A. MONTANARI, *Generalization error of random feature and kernel methods: Hypercontractivity and kernel matrix concentration*, Applied and Computational Harmonic Analysis, 59 (2022), pp. 3–84, <https://doi.org/https://doi.org/10.1016/j.acha.2021.12.003>, <https://www.sciencedirect.com/science/article/pii/S1063520321001044>. Special Issue on Harmonic Analysis and Machine Learning.
- [29] I. MEZIĆ, *Spectral properties of dynamical systems, model reduction and decompositions*, Nonlinear Dynamics, 41 (2005), p. 309–325, <https://doi.org/https://doi.org/10.1007/s11071-005-2824-x>.
- [30] I. MEZIĆ, *Spectrum of the Koopman operator, spectral expansions in functional spaces, and state-space geometry*, Journal of Nonlinear Science, 30 (2020), p. 2091–2145, <https://doi.org/https://doi.org/10.1007/s00332-019-09598-5>.
- [31] T. MISIAKIEWICZ AND A. MONTANARI, *Six lectures on linearized neural networks*, Journal of Statistical Mechanics: Theory and Experiment, 2024 (2024), p. 104006, <https://doi.org/10.1088/1742-5468/ad292a>, <https://dx.doi.org/10.1088/1742-5468/ad292a>.
- [32] N. H. NELSEN AND A. M. STUART, *Operator learning using random features: A tool for scientific computing*, SIAM Review, 66 (2024), pp. 535–571, <https://doi.org/10.1137/24M1648703>, <https://doi.org/10.1137/24M1648703>, <https://arxiv.org/abs/https://doi.org/10.1137/24M1648703>.
- [33] A. RAHIMI AND B. RECHT, *Random features for large-scale kernel machines*, in Advances in Neural Information Processing Systems, vol. 20, Curran Associates, Inc., 2007, https://proceedings.neurips.cc/paper_files/paper/2007/file/013a006f03dbc5392effeb8f18fda755-Paper.pdf.
- [34] A. RAHIMI AND B. RECHT, *Uniform approximation of functions with random bases*, in 2008 46th Annual Allerton Conference on Communication, Control, and Computing, 2008, pp. 555–561, <https://doi.org/10.1109/ALLERTON.2008.4797607>.
- [35] A. RUDI AND L. ROSASCO, *Generalization properties of learning with random features*, in Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, Red Hook, NY, USA, 2017, Curran Associates Inc., p. 3218–3228.
- [36] B. SCHÖLKOPF, R. HERBRICH, AND A. J. SMOLA, *A generalized representer theorem*, in Computational Learning Theory, Berlin, Heidelberg, 2001, Springer Berlin Heidelberg, pp. 416–426.
- [37] B. SCHÖLKOPF AND A. J. SMOLA, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*, MIT Press, Cambridge, MA, USA, 2001.
- [38] Y. SUN, A. GILBERT, AND A. TEWARI, *But how does it work in theory? linear SVM with random features*, in Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., 2018, https://proceedings.neurips.cc/paper_files/paper/2018/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.
- [39] W. TANG, *Koopman operator in the weighted function spaces and its learning for the estimation of Lyapunov and Zubov functions*, arXiv: 2410.00223, (2024), <https://arxiv.org/abs/2410.00223>.
- [40] B. UMATHE AND U. VAIDYA, *Spectral Koopman method for identifying stability boundary*, IEEE Control Systems Letters, 8 (2024), pp. 247–252, <https://doi.org/10.1109/LCSYS.2023.3347396>.
- [41] U. VAIDYA, *Spectral analysis of Koopman operator and nonlinear optimal control*, in 2022 IEEE 61st Conference on Decision and Control (CDC), 2022, pp. 3853–3858, <https://doi.org/10.1109/CDC51059.2022.9993265>.
- [42] U. VAIDYA, *When Koopman meets Hamilton and Jacobi*, arXiv: 2504.07346, (2025), <https://arxiv.org/abs/2504.07346>.
- [43] M. O. WILLIAMS, C. W. ROWLEY, AND I. G. KEVREKIDIS, *A kernel-based method for data-driven koopman spectral analysis*, Journal of Computational Dynamics, 2 (2015), pp. 247–265, <https://doi.org/10.3934/jcd.2015005>, <https://www.aims sciences.org/article/id/ce535396-f8fe-4aa1-b6de-baaf986f6193>.
- [44] G. YEHUDAI AND O. SHAMIR, *On the power and limitations of random features for understanding neural networks*, in Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., 2019, https://proceedings.neurips.cc/paper_files/paper/2019/file/5481b2f34a74e427a2818014b8e103b0-Paper.pdf.